CPSC 416: Design Project 4 - PAXOS

Background

Context: We are tasked with creating a fault-tolerant PAXOS key value store with multi-server replication. This service will maintain consistency across its multiple (2+) nodes, and ensure resilience to network partitions - in-turn sacrificing availability. In-particular , unlike our simple Primary/Backup implementation we will be able to make progress here as long as a majority of nodes are in agreement. We follow requirements specified as part of the lab instructions. We will discuss the general ideas, assumptions, failure models, invariants, protocol sketch, and observability plan of this design.

General idea: At its core, our implementation involves multiple PaxosServers which are themselves responsible for managing the system, and acting as request handling nodes by coordinating among themselves via a two phased protocol. It is assumed that the client will respect the rules of our system, as in they will not act in a "byzantine" fashion (maliciously finding hacks, or bypassing security measures), and interact with the servers via standard public APIs for managing key-value stores. Our key-value store will be an append-only store, this is to ensure ease of consistency. Thus, we allow the general APIs of get, put, update but not delete. Each PaxosServer is able to simultaneously perform the roles of Replica, Follower, and Leader. That is, it will serve as a Log replica in case other nodes fail, act as a follower to vote on "majorities" or accepted logs, and if there is no other leader (we follow single leader constraint) it can bid for the leader position. Leaders can act in all three roles, and they are responsible for ensuring consensus among our nodes. Client requests will be received by all nodes, but it is the leader that is responsible for determining which request was accepted by the majority in a particular slot via Follower-Leader Acks and then communicating that updated log entry to all nodes via Leader-Confirmation Log messages. Upon receiving a client request, all nodes except the leader will ignore the request - potentially forwarding to the leader if necessary. Once the leader receives that request it will "propose" to the other nodes that this entry should be in log slot k, if a majority reply with a successful ack that will be committed, communicated, and replicated across the system. Upon leader failure, a new leader can be elected from available follower nodes.

Failure Model: We focus on the "timing failure" model. This is because in our system, it is impossible to distinguish between servers that have crashed or servers that are temporarily unavailable. We are not guaranteed a reliable system, our messages can be reordered, lost, or delayed, we are unsure if they will, or when they will reach the desired endpoint and when we will get back a response. Thus, we bound our system on response timing. Specifically, a PaxosServer leader will send heartbeats to its followers. Upon two missed heartbeats the followers can elect to become leader. We will also bind response time between other protocols, for example the client and our servers.

Assumptions:

- The set of servers we work with is fixed, old servers that fail will not be replaced.
- Network is unreliable, messages may be delayed, lost, reordered.
- Servers may become unrecoverable, we do not implement/care about server recovery in this lab.
- Primary and backup servers operate in a single-threaded environment, that is one-at a time. Furthermore, there is only ever one primary or backup server active at any one point in time.
- Data is stored in volatile memory, that is if all servers crash we cannot recover.
- If primary fails with no backup, we will not go back to uninitialized state (test spec).

Desired Properties from Doc:

- We assume a single-threaded system where at most one-command is run for a particular server at a particular moment in time.
- For each logslot, denoted k, at most one value is chosen.
- Eventually consistent system, the system can make progress when the majority of nodes can communicate and will eventually make progress.
- Message and time handlers should be deterministic.
- Service should be consistent.
- Service should be resilient to network partitions.
- For the purposes of garbage collection, to compact the logs, a particular slot is only cleared when it has been executed on all servers.
- Operations should execute linearizably, that is one-at-a-time we can order them sequentially and reconstruct the state consistently across the multiple servers.
- Operations should provide exactly-once semantics, that is repeated instructions (perhaps from a duplicate request) do not cause changes in the system.

Deduced Invariants:

- Single chosen value per slot
- Acceptors never accept proposals for ballots lower than the last they accepted.
- Although acceptance can happen in any order, execution must happen in the correct order such that operations <k are done before k.
- Liveness based on eventual completion.

Protocol Design

It proceeds as follows:

Basic Information:

- All Paxos Servers know the address of each other, as this is a fixed server list.

Phase 1 (Initiation):

1. Before proceeding with routine phase 2 activity (which deals with client request), out of our Paxos Servers we must select a leader. This will proceed normally, as our PaxosServers will see they have not received a Heartbeat from the leader despite two Heartbeat Intervals passing and will send a prepare(Ballot) request to all other servers. Ballot in the form {Round, LeaderNumber} ie. {1, S1} in this case (as an example).
2. Whichever leader first receives a majority of promises becomes the leader. Promises are in the form (Ballot, pvalues). The Ballot must match what the leader proposer has sent over (ie. {1, S1}) as a form of Ack that it has a vote to become the new leader. The pvalues are a list of non-garbage collected (as gc collected commands are already universal) commands accepted by that acceptor, in the form (Ballot at that point, Slot, Value) this specifies that value written to that slot for that Paxos Server and with the ballot/priority it had.
    a. Note a node is automatically considered to vote for itself, to avoid the “server does not send itself a message” problem.
3. Once a leader is elected it must consolidate the pvalues received, it will create an updated log by proposing pvalues to all servers in the form propose(Ballot, slot, value) but it will only propose pvalues that have the highest ballot if two conflict. (Essentially follow phase 2)
    a. At the same time, we will be sending Heartbeats to all known servers every heartbeat interval.

4. The leader will be pre-empted if it receives a higher ballot response from a follower than it currently possesses at which point it will itself become a follower (go into Phase 2).

Phase 2 (Routine):

1. Client makes an API request to all known servers (simple as they are fixed).
2. Only the primary server is able to propose requests to other PaxosServers in the form Propose(Ballot, slot, value). It does so and will wait for a majority response (ie. accept-ack) for a particular Ballot/Slot/Value.
    a. Followers may forward the request, but do not execute.
3. Once it receives a majority response it will broadcast this message via an Accept(Ballot, Slot, Value) to all PaxosServers.
    a. Each paxos server stores this command in its own log, although paxos servers can accept out of order log commands it will only execute in order.
    b. Note, if a server misses two heartbeat intervals from a leader it will propose itself as a leader (see Phase 1).
    c. For garbage collection, on response we can include lastExecuted. The leader can keep track of a set of lastExecuted operations for all servers, and G-C logs that are prior to the minimum lastExecuted operation thereby compacting the log.
4. Leader returns once the command is chosen.
    a. If the command is something simp
5. Primary interprets response, depending on response it will log and then update.
6. Primary sends the client its response.

Failure Cases Communication:

- Client:
    - Fails to receive any response from the server it contacts.
        - Solution: Client retries the request with exponential backoff.
    - Receives Failure Response:
        - Operation Rejected:
            - Solution: No issue here, this can happen if the API request was invalid.
        - Busy:
            - Depending on the decided complexity, it could be the case that the server is currently busy responding to another request.
            - Solution: We have to decide whether to buffer the requests on the server side or reject outright. I suggest buffering client requests, we will still respond busy when we reach a set limit. Thus the client should retry.
        - Internal Error:
            - This response will be received if there is some inconsistency in the system that prevents the server from executing the request, this is not the clients responsibility. We do not expect this to be possible in our system, and have no recovery for it.
- Servers:
    - PaxosServer receives Proposal:
        - Upon receiving a proposal we will ensure the ballot of that proposal is at least as large as any ballot we have previously seen, rejecting otherwise (with StaleBallot as a reason).
        - We will also ensure any request has a slot that is >= to our lowestExecuted slot. Ie. we will not overwrite garbage collected slots.

        - We will ensure duplicate commands are not written.
    - PaxosServer sending Proposal/Waiting for Acks:
        - The leader (or potential leaders) will retry both Phase 1 and Phase 2 if a timeout is exceeded on waiting for response acks.
    - Follower is lagging behind:
        - Followers will send periodic lastExecuted, lastCompleteLogged messages to the leader. The leader will send missing messages, that is messages between the lastLogged and lastExecuted to that follower node so it can catch up in execution in case it is missing those messages. For example if leader receives {4, 7} and currently we are at slot 10. Leader will attempt to send slot 8 value to this follower so it can catch up eventually and the lastExecuted can update. Note the lastCompleteLogged is the largest number with all prior log entries in the log (ie. even if we had seen command for slot 9 we would return 9 if slot 8 was missing)

Other Failure Cases:

- Network is partitioned:
    - Case 1: Majority is still in partition, there is no issue we will be eventually progressing.
    - Case 2: Majority is not in a partition, progress will stop.
- Out of order requests:
    - Servers must reject stale ballot commands and avoid updating slots that have already been executed/written into log and agreed upon (by non-higher ballots).

Observability Plan

We will log lightweight protocol and request metrics to help debug correctness and liveness issues (especially retries, preemption, and slow progress). The main signals we care about are: client request outcomes, Paxos progress (accept/decision), leader changes, and catch-up / garbage collection behavior. We also track basic latency so we can spot stalls caused by quorum delays or repeated leader contention.

- leader changes / preemption: (node, oldBallot -> newBallot, becameLeader|steppedDown, reason)
- phase1 summary: (node, ballot, quorumReached, pvaluesCount)
- phase2 accepts: (node, ballot, slot, valueSummary, quorumAcked)
- log decisions learned: (node, slot, valueSummary, learnedFrom)
- log execution: (node, slot, op, clientId, reqId, resultSummary)
- client ops: (node, clientId, reqId, op, slot?, outcome=ok|retry|redirect|timeout, latencyMs)
- duplicates / dedup: (node, clientId, reqId, duplicate=true, priorResultReturned)
- heartbeats: (node, leaderBallot, leaderId, lastExecuted, gcSlot)

- catch-up sends: (leaderNode, followerNode, fromSlot, toSlot)
- garbage collection: (node, oldFirstNonCleared -> newFirstNonCleared, gcSlot)
- rejects / stale messages: (node, msgType, reason=staleBallot|oldSlot|preempted|notLeader, localBallot, msgBallot, slot?)